

Fundamentals of Deep Learning and Programming

Lecture 8: Introduction to Natural Language Processing (NLP)(2) and Attention Mechanism

Fall Semester 2025

Adjunct Professor: Donghoon Kang

Counter class inside collection module

```
from collections import Counter
```

- Counter class is like a dictionary that counts how many times each item appears.

Counting words in a list

```
from collections import Counter

counter = Counter()          # create an empty counter
counter.update(["cat", "dog", "cat"])

print(counter)
```

```
Counter({'cat': 2, 'dog': 1})
```

counter.update() adds counts to the counter.

Updating multiple times

```
from collections import Counter

counter = Counter()          # create an empty counter
counter.update(["cat", "dog", "cat"])
counter.update(["cat", "cow"])

print(counter)
```

```
Counter({'cat': 3, 'dog': 1, 'cow': 1})
```

Counting words

```
from collections import Counter

tokens = ["cat", "dog", "cat", "bird", "cat", "dog"]
counter = Counter(tokens)

print(counter.most_common()) # sorts them by frequency.
[('cat', 3), ('dog', 2), ('bird', 1)]
```

`counter.most_common()`
Return all items sorted by frequency (high → low)

`counter.most_common(n)` Return only the top `n` items

```
from collections import Counter

text = "I like dogs and I like cats and I like animals"
words = text.lower().split() # very simple tokenization

counter = Counter(words)

for word, freq in counter.most_common():
    print(word, "→", freq)
```

```
i → 3
like → 3
and → 2
dogs → 1
cats → 1
animals → 1
```

```
from collections import Counter

animals = "catdogcatcat"
counter = Counter(animals)

print(counter.most_common(1))
```

```
[('c', 3)]
```


Python List

```
text = "I LIKE CATS"
print(text.lower())
print(text.lower().split())
```

```
i like cats
['i', 'like', 'cats']
```

```
toks = ["How", "are", "you?"]
text = ':'.join(toks)
print(text)
```

```
text = ' '.join(toks)
print(text)
```

```
How:are:you?
How are you?
```

Python Dictionary

```
person = {"name": "Minsoo", "age": 25}
```

```
print(person.get("name")) # Minsoo
print(person.get("job")) # None (because "job" doesn't exist)
print(person.get("job", "Not Allowed")) # default value is returned
```

```
Minsoo
None
Not Allowed
```

```
person = {"name": "Minsoo", "age": 25}
```

```
print(person["name"])
print(person["job"])
```

```
Traceback (most recent call last):
  File "D:\dev\python\myTorch\pytorch\lec\c07\python test.py", line 8, in <module>
    print(person["job"])
KeyError: 'job'
```

Python List Comprehension

```
from collections import Counter

corpus_ids = [[5, 0, 2, 4], [5, 1, 2, 6], [5, 0, 3, 4], [5, 1, 3, 6]]

words = []

for sent in corpus_ids:
    for w in sent:
        words.append(w)

print("words: ", words)
```

```
words: [5, 0, 2, 4, 5, 1, 2, 6, 5, 0, 3, 4, 5, 1, 3, 6]
```

```
# Count how many times each word ID appears
```

```
freq_1 = Counter(words)
print("freq_1: ", freq_1)
```

```
# nested list comprehension
```

```
freq_2 = Counter([w for sent in corpus_ids for w in sent])
print("freq_2: ", freq_2)
```

```
freq_1: Counter({5: 4, 0: 2, 2: 2, 4: 2, 1: 2, 6: 2, 3: 2})
freq_2: Counter({5: 4, 0: 2, 2: 2, 4: 2, 1: 2, 6: 2, 3: 2})
```

Full NLP pipeline usually looks like this

1. Tokenize
2. Build vocabulary
3. Encode text into token IDs
4. → Word embeddings (required for ML models)
5. → Feed embeddings into a model (Transformer, LSTM, etc.)
6. Train
7. Decode prediction back to text

Word2Vec is a method in Natural Language Processing (NLP) for turning words into vectors of numbers. These vectors are called **word embeddings**.

The main idea is simple:

Words that appear in similar contexts should have similar vectors.

For example, after training Word2Vec on a large text corpus, we might get:

“cat” $\rightarrow$ $[0.31, -0.22, 0.87, \dots]$, “dog” $\rightarrow$ $[0.29, -0.20, 0.83, \dots]$.

The vectors for “cat” and “dog” are close to each other in the vector space, because they are semantically similar words.

Why do we need Word Embedding ?

Neural networks and other machine learning models work with numbers, not with text. So we must convert each word into a numerical representation.

Before Word2Vec, common approaches were:

- **One-hot vectors**: very long, mostly zeros, no notion of meaning.
- **Bag-of-words**: counts word occurrences, but ignores word order and deeper semantics.

Word2Vec solves these problems by learning:

- **dense** vectors (low-dimensional, e.g. 100 or 300 dimensions),
- **meaningful** vectors (geometry reflects similarity and relationships).

Core Idea : Context Predicts Meaning

Word2Vec is based on a famous idea in linguistics, often called the *distributional hypothesis*:

“You shall know a word by the company it keeps.”

This means that the **context** around a word gives strong clues about its meaning.
For example:

- “The cat drinks milk.”
- “The dog drinks water.”

Because “cat” and “dog” appear in similar sentences and with similar surrounding words, Word2Vec will learn similar vectors for them.

Two Main Architectures: CBOW and Skip-gram

Word2Vec usually refers to two related neural network models:

1. CBOW (Continuous Bag of Words)

Predict the center word from its context words. For example, from “the ____ drinks milk”, predict “cat”.

2. Skip-gram

Predict the context words from a center word. For example, from “cat”, predict words like “the” and “drinks”.

Both models are small neural networks with one hidden layer. The important part is that during training, the network learns good embeddings for each word.

What Does Word2Vec Learn ?

Mathematically, Word2Vec learns an **embedding matrix**:

$$W \in \mathbb{R}^{V \times D},$$

where:

- V is the vocabulary size (number of distinct words),
- D is the embedding dimension,
- each row W_w is the embedding vector for word w .

This matrix is the main result of training. We do not usually use the whole Word2Vec model later, we just use the learned matrix W .

Example of Semantic Structure

One of the famous properties of Word2Vec embeddings is that they capture intuitive relationships between words. For example:

$$\text{vector}(\text{"king"}) - \text{vector}(\text{"man"}) + \text{vector}(\text{"woman"}) \approx \text{vector}(\text{"queen"}).$$

This shows that the model has captured some notion of gender and royalty in the geometry of the vectors.

Training Trick: Negative Sampling

Computing the full probability distribution over a large vocabulary can be expensive. Word2Vec uses a technique called **negative sampling** to make training efficient.

Informally, for each correct (center, context) pair, the model:

- rewards the pair (makes their vectors more similar),
- and chooses a few random “negative” words and pushes their vectors away.

This allows Word2Vec to train on very large corpora quickly.

- Negative sampling makes training efficient while still learning meaningful relationships.

How Word2Vec is Used in Practice

Once Word2Vec has been trained on a large text corpus, we can:

- use the word vectors as input features to other models (e.g., classifiers, RNNs, Transformers),
- measure similarity between words using cosine similarity,
- explore relationships and analogies between words.

Word2Vec is a way to convert words into useful numerical vectors that capture meaning.

What is Word2Vec ?

Word2Vec is a method to turn words into vectors of numbers (embeddings). The main idea is:

If two words appear in similar contexts, their vectors should become similar.

For example, after training on many sentences, we might get:

“cat” $\rightarrow$ $[0.3, -0.2, 0.8, \dots]$, “dog” $\rightarrow$ $[0.29, -0.18, 0.75, \dots]$.

We will focus on a very small toy world with just four words:

$\{\text{cat}, \text{dog}, \text{milk}, \text{water}\},$

and use this to illustrate the math and intuition.

We use the **row convention**: each word embedding is stored in a row of a matrix.

- In the example, “cat” becomes associated with “milk”, and “dog” with “water”.

Tiny Vocabulary and One-Hot Representation

Assume our tiny vocabulary is:

$$0 \rightarrow \text{"cat"}, \quad 1 \rightarrow \text{"dog"}, \quad 2 \rightarrow \text{"milk"}, \quad 3 \rightarrow \text{"water"}.$$

So the vocabulary size is $V = 4$.

Each word can be represented as a one-hot **row** vector of length V :

$$\text{"cat"} \Rightarrow x_{\text{cat}} = [1, 0, 0, 0],$$

$$\text{"dog"} \Rightarrow x_{\text{dog}} = [0, 1, 0, 0],$$

$$\text{"milk"} \Rightarrow x_{\text{milk}} = [0, 0, 1, 0],$$

$$\text{"water"} \Rightarrow x_{\text{water}} = [0, 0, 0, 1].$$

In practice, we rarely create these one-hot vectors explicitly. Instead, we use the index (0, 1, 2, or 3) to select the correct row from an embedding matrix.

Embedding Matrices (input and output)

Word2Vec (Skip-gram) uses two embedding matrices:

$$W \in \mathbb{R}^{V \times D}, \quad W' \in \mathbb{R}^{V \times D},$$

where:

- V is the vocabulary size (here $V = 4$),
- D is the embedding dimension,
- each row W_w is the **input embedding** of word w ,
- each row W'_w is the **output embedding** of word w .

For a center word w_c , its input embedding is:

$$v_c = W_{w_c}.$$

For a context word w_o , its output embedding is:

$$v'_o = W'_{w_o}.$$

Small Toy Example with Dimension $D = 2$

To make things concrete, let us choose a very small embedding dimension $D = 2$. Suppose that at some point during training, the input embedding matrix W and output embedding matrix W' look like this:

$$W = \begin{bmatrix} W_{\text{cat}} \\ W_{\text{dog}} \\ W_{\text{milk}} \\ W_{\text{water}} \end{bmatrix} = \begin{bmatrix} 0.20 & 0.10 \\ 0.25 & 0.05 \\ -0.30 & 0.40 \\ -0.28 & 0.35 \end{bmatrix}, \quad W' = \begin{bmatrix} W'_{\text{cat}} \\ W'_{\text{dog}} \\ W'_{\text{milk}} \\ W'_{\text{water}} \end{bmatrix} = \begin{bmatrix} 0.10 & 0.30 \\ 0.20 & 0.20 \\ -0.40 & 0.50 \\ -0.35 & 0.45 \end{bmatrix}.$$

Then the input embeddings are:

$$v_{\text{cat}} = W_{\text{cat}} = [0.20, 0.10], \quad v_{\text{dog}} = W_{\text{dog}} = [0.25, 0.05],$$

$$v_{\text{milk}} = W_{\text{milk}} = [-0.30, 0.40], \quad v_{\text{water}} = W_{\text{water}} = [-0.28, 0.35],$$

and the output embeddings are:

$$v'_{\text{cat}} = W'_{\text{cat}} = [0.10, 0.30], \quad v'_{\text{dog}} = W'_{\text{dog}} = [0.20, 0.20],$$

$$v'_{\text{milk}} = W'_{\text{milk}} = [-0.40, 0.50], \quad v'_{\text{water}} = W'_{\text{water}} = [-0.35, 0.45].$$

Note that W and W' are not derived from each other. They are separate trainable matrices learned during training.

Small Corpus and Training Pairs

Imagine a tiny corpus with sentences such as:

- “the cat drinks milk”
- “the dog drinks water”
- “the cat likes milk”
- “the dog likes water”

If we only pay attention to the four words {cat, dog, milk, water} and use a small context window, we might extract Skip-gram training pairs like:

(center: “cat”, context: “milk”), (center: “dog”, context: “water”),

Thus, the model should learn that “cat” is strongly associated with “milk”, and “dog” is strongly associated with “water”. Depending on the corpus, it may also learn that both “milk” and “water” are *drinks*, and that “cat” and “dog” are both *animals*.

Dot Products for Some Pairs

Consider the pair (center = “cat”, context = “milk”). We have:

$$v_{\text{cat}} = [0.20, 0.10], \quad v'_{\text{milk}} = [-0.40, 0.50].$$

The dot product is:

$$v_{\text{cat}} \cdot (v'_{\text{milk}})^T = 0.20 \cdot (-0.40) + 0.10 \cdot 0.50 = -0.08 + 0.05 = -0.03.$$

Now consider (center = “dog”, context = “water”):

$$v_{\text{dog}} = [0.25, 0.05], \quad v'_{\text{water}} = [-0.35, 0.45],$$

so

$$v_{\text{dog}} \cdot (v'_{\text{water}})^T = 0.25 \cdot (-0.35) + 0.05 \cdot 0.45 = -0.0875 + 0.0225 = -0.065.$$

During training, the model will adjust the rows of W and W' to make the dot products for *true* (center, context) pairs larger, and the dot products for *false* pairs smaller.

For example, if (“cat”, “water”) is rarely a real pair in the data, then the model will push

$$v_{\text{cat}} \cdot (v'_{\text{water}})^T$$

to be relatively small, compared to

$$v_{\text{cat}} \cdot (v'_{\text{milk}})^T.$$

Skip-gram Probability (Conceptual)

In the full softmax version, the probability of a context word w_o given a center word w_c is:

$$p(w_o | w_c) = \frac{\exp(v_c \cdot (v'_o)^T)}{\sum_{w=1}^V \exp(v_c \cdot (v'_w)^T)}.$$

In our small example with $V = 4$ and center word “cat”, the denominator sums over {cat, dog, milk, water}:

$$\sum_{w \in \{\text{cat}, \text{dog}, \text{milk}, \text{water}\}} \exp(v_{\text{cat}} \cdot (v'_w)^T).$$

The training objective encourages $p(\text{“milk”} | \text{“cat”})$ to be high, and similarly $p(\text{“water”} | \text{“dog”})$ to be high.

Negative Sampling (Intuition)

The training objective encourages $p(\text{“milk”} \mid \text{“cat”})$ to be high, and similarly $p(\text{“water”} \mid \text{“dog”})$ to be high.

- one positive context word (e.g., “milk” for “cat”),
- a few negative words sampled from the vocabulary (e.g., “dog”, “water” for “cat”).

The negative sampling loss for one center–context pair (w_c, w_o) is:

$$\mathcal{L} = -\log \sigma(v_c \cdot (v'_o)^T) - \sum_{k=1}^K \log \sigma(-v_c \cdot (v'_{n_k})^T),$$

where $\sigma(x) = \frac{1}{1 + e^{-x}}$ is the sigmoid, and v'_{n_k} are the output embeddings of negative words.

In our toy example, if (“cat”, “milk”) is a positive pair, we might sample “dog” and “water” as negative words. Then training:

- increases $v_{\text{cat}} \cdot (v'_{\text{milk}})^T$,
- decreases $v_{\text{cat}} \cdot (v'_{\text{dog}})^T$ and $v_{\text{cat}} \cdot (v'_{\text{water}})^T$.

What Gets Updated

For each training step involving a center word w_c and context word w_o (with some negative samples), only a few rows are updated:

- W_{w_c} (the center word row in W),
- W'_{w_o} (the context word row in W'),
- $W'_{w_{n_k}}$ for each negative sample.

All other rows remain unchanged in that step. Over many steps, this leads to meaningful embeddings.

Final Word Embeddings

After training, we usually take:

$$v_w = W_w$$

as the final embedding vector for each word w (the row of W).

Sometimes, people also combine W and W' by averaging:

$$v_w = \frac{W_w + W'_w}{2}.$$

In our tiny world, this would give us embeddings for “cat”, “dog”, “milk”, and “water” that reflect their roles in the training sentences: cats with milk, dogs with water, and so on.

Each word is represented by a row in the embedding matrices W and W' .

The Skip-gram model learns these matrices so that center words are good at predicting their context words.

Why Negative Sampling is Used Instead of Full Softmax ?

In a full softmax model, the probability of a context word given a center word is

$$P(\text{context} \mid \text{center}) = \frac{\exp(v_c \cdot v_o)}{\sum_{w=1}^V \exp(v_c \cdot v_w)},$$

where v_c is the embedding of the center word, v_o is the embedding of the context word, and V is the vocabulary size.

Why Negative Sampling is Used Instead of Full Softmax ?

The problem is the denominator:

$$\sum_{w=1}^V \exp(v_c \cdot v_w),$$

which requires a dot product with *every* word in the vocabulary. If V is very large (e.g. hundreds of thousands or millions of words), this becomes computationally infeasible for each training step.

Why Negative Sampling is Used Instead of Full Softmax ?

Negative sampling replaces this expensive full softmax with a much simpler binary classification task. Instead of computing probabilities over the entire vocabulary, the model asks:

“Is this (center, context) pair real or fake?”

For each real (positive) pair, the algorithm samples a small number K of negative (fake) context words. The model then computes:

- 1 dot product for the positive pair, and
- K dot products for the negative pairs,

for a total of $1 + K$ dot products, instead of V . This makes training much faster and more scalable while still learning high-quality word embeddings.

1. Word2Vec computes 1 positive dot product and K negative dot products per training pair.
2. It is not too small because the algorithm **generates a huge number of positive pairs** from the sliding window over the corpus.

What is n-gram ?

- Consider a word's count by the total number of words (tokens) in the training text (corpus):

$$P_{MLE}(w) = \frac{Count(w)}{N}$$

where w is the word, $Count(w)$ is its frequency, and N is the total number of words.

The major problem with this approach is data sparsity:

- Any word that does not appear in the training corpus will be assigned a probability of zero.
- This is problematic because it implies that these "unseen" words are impossible to encounter in real-world text, which is an incorrect assumption.

What is n-gram ?

An n-gram is a sequence of (n) consecutive items taken from text.

These items can be:

- words
- characters
- subword units

N-grams are used to capture local context in language.

Consider the sentence”

[the cat drinks milk]

Unigrams (1-grams) : [(the), (cat), (drinks), (milk)]

Bigrams (2-grams) : [(the cat), (cat drinks), (drinks milk)]

Trigrams (3-grams) : [(the cat drinks), (cat drinks milk)]

Why n-grams are useful ?

N-grams help model:

- which words tend to appear together
- local linguistic patterns
- simple probabilistic language models

How the Sampling Table Relates to Probabilities

Negative sampling uses a smoothed unigram distribution to choose negative words. The probability of sampling a word w is defined as

$$P(w) = \frac{f(w)^{0.75}}{\sum_u f(u)^{0.75}},$$

where $f(w)$ is the frequency of word w in the corpus.

How the Sampling Table Relates to Probabilities

$$P(w) = \frac{f(w)^{0.75}}{\sum_u f(u)^{0.75}},$$

where $f(w)$ is the frequency of word w in the corpus.

In practice, instead of computing this probability from scratch every time, Word2Vec builds a large *sampling table*. The idea is:

1. For each word w , compute its weight $f(w)^{0.75}$.
2. Convert this into a number of repetitions in the table, for example,
$$\text{repetitions}(w) \approx f(w)^{0.75} \times \text{scale}.$$
3. Construct a list (table) that contains each word ID repeated according to this number of repetitions.
4. To draw a negative sample, simply pick a random element from this table (e.g. using `random.choice`).

Why 0.75 is a good exponent for smoothing frequencies

The exponent 0.75 in the distribution $f(w)^{0.75}$ controls how much we *smooth* the raw frequencies.

Case 1: No smoothing ($\alpha = 1.0$)

If we use $f(w)^1 = f(w)$, the distribution is proportional to the raw frequencies. Very common words like “the”, “of”, and “and” will dominate the negative samples. The model will see these words too often as negatives, which is not very informative.

Case 2: Strong smoothing ($\alpha = 0.5$)

If we use something like $f(w)^{0.5}$, we reduce the influence of frequent words aggressively. In this case, rare words may be sampled too often as negatives. This can make training noisy and less stable.

Why 0.75 is a good exponent for smoothing frequencies

Case 3: Balanced smoothing ($\alpha = 0.75$)

Using $f(w)^{0.75}$ is a compromise:

- Very frequent words are down-weighted, so they do not dominate the negative samples.
- However, they are not down-weighted so aggressively that they almost never appear.
- Rare words still get sampled, but not excessively.

Empirically, experiments in the Word2Vec literature and implementations show that $\alpha = 0.75$ works very well: it produces good-quality embeddings and stable training. The value 0.75 is therefore best understood as an empirically chosen exponent that balances the roles of frequent and rare words.

Why Negative Sampling is Used Instead of Full Softmax ?

- Full softmax is too expensive for large vocabularies, so negative sampling is used instead.
- The sampling table is a practical way to approximate the smoothed distribution $P(w) \propto f(w)^{0.75}$ by repeating word IDs according to their weights.
- The exponent 0.75 provides a good balance: it reduces the dominance of extremely frequent words without over-emphasizing rare words, leading to better embeddings in practice.

Simple Word2Vec code

simple_word2vec.py

```
import random
from collections import Counter
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
# =====
```

```
# 1. Simple corpus
```

```
# =====
```

```
# We focus on four words: cat, dog, milk, water
```

```
corpus = [
    "the cat drinks milk",
    "the dog drinks water",
    "the cat likes milk",
    "the dog likes water",
]
```

```
# =====
```

```
# 2. Build vocabulary
```

```
# =====
```

```
tokens = " ".join(corpus).split()
```

```
vocab = sorted(set(tokens))
```

```
# e.g. ['cat', 'dog', 'drinks', 'likes', 'milk', 'the', 'water']
```

```
word2id = {w: i for i, w in enumerate(vocab)}
```

```
id2word = {i: w for w, i in word2id.items()}
```

```
print("word2id:", word2id)
```

```
print("id2word:", id2word)
```

```
word2id: {'cat': 0, 'dog': 1, 'drinks': 2, 'likes': 3, 'milk': 4, 'the': 5, 'water': 6}
```

```
id2word: {0: 'cat', 1: 'dog', 2: 'drinks', 3: 'likes', 4: 'milk', 5: 'the', 6: 'water'}
```

```
vocab_size = len(vocab)
```

```
print("Vocab:", vocab)
```

```
print("Vocab size:", vocab_size)
```

```
Vocab: ['cat', 'dog', 'drinks', 'likes', 'milk', 'the', 'water']
```

```
Vocab size: 7
```

```
# Convert each sentence into a list of word IDs
```

```
corpus_ids = [[word2id[w] for w in sent.split()] for sent in corpus]
```

corpus_ids

```
[[5, 0, 2, 4], [5, 1, 2, 6], [5, 0, 3, 4], [5, 1, 3, 6]]
```

Simple Word2Vec code

simple_word2vec.py

```
# =====
# 3. Build Skip-gram (center, context) pairs
# =====
def make_skipgram_pairs(corpus_ids, window_size=1):
    """
    For each word in each sentence, create pairs:
    (center_word, context_word) within a given window size.
    """
    pairs = [] # an empty list to store all (center, context) pairs
    for sent in corpus_ids:
        for i, center in enumerate(sent):
            # context window: [i - window_size, ..., i + window_size]
            for j in range(max(0, i - window_size), min(len(sent), i + window_size + 1)):
                if j == i:
                    continue # skip the center word itself
                context = sent[j]
                pairs.append((center, context))
    return pairs

pairs = make_skipgram_pairs(corpus_ids, window_size=1)
print("Skip-gram pairs (id):", pairs)
print("Skip-gram pairs (words):", [(id2word[c], id2word[o]) for c, o in pairs])
```

```
Skip-gram pairs (id): [(5, 0), (0, 5), (0, 2), (2, 0), (2, 4), (4, 2), (5, 1), (1, 5), (1, 2), (2, 1), (2, 6), (6, 2), (5, 0), (0, 5), (0, 3), (3, 0), (3, 4), (4, 3), (5, 1), (1, 5), (1, 3), (3, 1), (3, 6), (6, 3)]
Skip-gram pairs (words): [('the', 'cat'), ('cat', 'the'), ('cat', 'drinks'), ('drinks', 'cat'), ('drinks', 'milk'), ('milk', 'drinks'), ('the', 'dog'), ('dog', 'the'), ('dog', 'drinks'), ('drinks', 'dog'), ('drinks', 'water'), ('water', 'drinks'), ('the', 'cat'), ('cat', 'the'), ('cat', 'likes'), ('likes', 'cat'), ('likes', 'milk'), ('milk', 'likes'), ('the', 'dog'), ('dog', 'the'), ('dog', 'likes'), ('likes', 'dog'), ('likes', 'water'), ('water', 'likes')]
```

```
# =====
# 4. Negative sampling helper
# =====
# Simple negative sampling: sample words according to frequency ^ 0.75
freq = Counter([w for sent in corpus_ids for w in sent])

# Build a list of word indices for sampling, weighted by freq^0.75
neg_sampling_table = []
for wid, f in freq.items():
    weight = f ** 0.75
    neg_sampling_table.extend([wid] * int(weight * 10)) # scale up for more resolution

print("freq.items(): ", freq.items())
print("neg_sampling_table: ", neg_sampling_table)
```

[illegible]

Simple Word2Vec code

simple_word2vec.py

```
def sample_negative(batch_pos_ids, num_negatives):  
    """  
    For each positive target id in batch_pos_ids,  
    sample 'num_negatives' different negative word ids (not equal to the positive one).  
    Returns a tensor of shape (batch_size, num_negatives).  
    """  
    batch_neg = []  
    for pos_id in batch_pos_ids:  
        negatives = []  
        while len(negatives) < num_negatives:  
            cand = random.choice(neg_sampling_table)  
            if cand != pos_id:  
                negatives.append(cand)  
        batch_neg.append(negatives)  
    return torch.tensor(batch_neg, dtype=torch.long)
```

Simple Word2Vec code

simple_word2vec.py

```
# =====  
# 5. Word2Vec Skip-gram model with negative sampling  
# =====  
class Word2VecSkipGram(nn.Module):  
    def __init__(self, vocab_size, embed_dim):  
        super().__init__()  
  
        # Embeddings for center words  
        self.in_embed = nn.Embedding(vocab_size, embed_dim)  
  
        # Embeddings for context words  
        self.out_embed = nn.Embedding(vocab_size, embed_dim)  
  
        # Initialize embeddings (optional but common)  
        # Fills the embedding weight tensor with values drawn from a uniform distribution in the range  
        nn.init.uniform_(self.in_embed.weight, -0.5 / embed_dim, 0.5 / embed_dim)  
        nn.init.uniform_(self.out_embed.weight, -0.5 / embed_dim, 0.5 / embed_dim)
```

```
nn.init.uniform_(self.in_embed.weight, -0.5 / embed_dim, 0.5 / embed_dim)
```

initializes the embedding matrix with values drawn from a uniform distribution. This follows the initialization strategy used in the original Word2Vec implementation.

Uniform Distribution Used

The function `uniform_` fills a tensor W with random values sampled from:

$$W \sim U(a, b),$$

where

$$a = -\frac{0.5}{\text{embed_dim}}, \quad b = +\frac{0.5}{\text{embed_dim}}.$$

Why This Range Is Used

1. Small Initial Values

Large initial embedding values can cause:

- exploding gradients,
- unstable optimization,
- excessively large dot products in early training.

Using a small range prevents these problems.

2. Scaling by Embedding Dimension

The dot product of two embeddings is:

$$v_c \cdot v_o = \sum_{i=1}^D v_{c,i} v_{o,i}.$$

If D is large and values are not small, the sum becomes too large.

Scaling by $\frac{1}{D}$ ensures stable dot products regardless of embedding dimension.

Simple Word2Vec code

simple_word2vec.py

```
def forward(self, center_ids, pos_ids, neg_ids):
    """
    center_ids: (batch,)
    pos_ids:    (batch,)
    neg_ids:    (batch, K)   where K = # negative samples
    """
    # Lookup embeddings
    center_embed = self.in_embed(center_ids)          # (batch, D)
    pos_embed    = self.out_embed(pos_ids)            # (batch, D)
    neg_embed    = self.out_embed(neg_ids)            # (batch, K, D) because each example has K negative words

    # Positive score: dot(center, pos)
    pos_score = torch.sum(center_embed * pos_embed, dim=1) # (batch,)
    pos_loss = F.logsigmoid(pos_score)                    # (batch,)
                # Large positive dot products → high probability → good for positive samples.

    # Negative score: dot(center, neg) for each negative
    # center_embed: (batch, D) → (batch, D, 1)
    # neg_embed: (batch, K, D)
    neg_score = torch.bmm(neg_embed, center_embed.unsqueeze(2)).squeeze(2) # (batch, K)
    neg_loss = F.logsigmoid(-neg_score).sum(dim=1)          # (batch,)

    # We want to MAXIMIZE pos_loss + neg_loss, so MINIMIZE negative of that
    loss = -(pos_loss + neg_loss).mean()
    return loss
```

Simple Word2Vec code

simple_word2vec.py

```
# =====  
# 6. Training loop  
# =====  
embed_dim = 8          # embedding dimension (can try 2, 8, 16, ...)  
num_negatives = 4       # number of negative samples  
batch_size = min(8, len(pairs))  
num_epochs = 2000       # training epochs  
lr = 0.01  
  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
  
model = Word2VecSkipGram(vocab_size, embed_dim).to(device)  
optimizer = torch.optim.Adam(model.parameters(), lr=lr)  
  
# Convert all pairs to tensors for easier batching  
all_centers = torch.tensor([c for c, o in pairs], dtype=torch.long)  
all_contexts = torch.tensor([o for c, o in pairs], dtype=torch.long)  
  
Loss: {loss.item():.4f}"
```


Simple Word2Vec code

simple_word2vec.py

```
for epoch in range(1, num_epochs + 1):
    # Sample a random mini-batch
    indices = torch.randint(0, len(pairs), (batch_size,))
    batch_centers = all_centers[indices].to(device)
    batch_pos      = all_contexts[indices].to(device)
    batch_neg      = sample_negative(batch_pos.tolist(), num_negatives).to(device)

    # Forward + Backward
    loss = model(batch_centers, batch_pos, batch_neg)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 200 == 0:
        print(f"Epoch {epoch}/{num_epochs} - Loss: {loss.item():.4f}")
```

```
Epoch 200/2000 - Loss: 1.5981
Epoch 400/2000 - Loss: 1.5177
Epoch 600/2000 - Loss: 1.5958
Epoch 800/2000 - Loss: 1.5795
Epoch 1000/2000 - Loss: 1.6512
Epoch 1200/2000 - Loss: 1.3453
Epoch 1400/2000 - Loss: 2.0184
Epoch 1600/2000 - Loss: 1.5621
Epoch 1800/2000 - Loss: 1.7174
Epoch 2000/2000 - Loss: 1.4892
```

Simple Word2Vec code

simple_word2vec.py

```
# =====  
# 7. Inspect learned embeddings  
# =====  
print("\nLearned input embeddings (in_embed):")  
for w in ["cat", "dog", "milk", "water"]:  
    wid = word2id[w]  
    vec = model.in_embed.weight[wid].detach().cpu().numpy()  
    print(f"{w:>6} → {vec}")
```

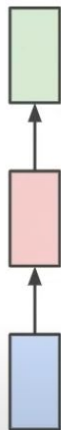
```
Learned input embeddings (in_embed):  
cat → [-0.43179485  1.0369823  1.81707   -0.525412   1.2287695  -0.77972484  
-0.4633337   0.48856345]  
dog → [-0.38001347  0.9518258  1.9261985  -0.5714616  1.2401487  -0.7929308  
-0.4508419   0.4589135 ]  
milk → [-1.6209836  -0.258975   0.77976435  0.7602426  1.1985468  -1.2426794  
-1.327505    1.7284908 ]  
water → [-1.6411728  -0.12531823  0.7720647  0.97278976  1.1164039  -1.1065408  
-1.3086174   1.6379796 ]
```

Full NLP pipeline usually looks like this

1. Tokenize
2. Build vocabulary
3. Encode text into token IDs
4. → Word embeddings (required for ML models)
5. → Feed embeddings into a model (Transformer, LSTM, etc.)
6. Train
7. Decode prediction back to text

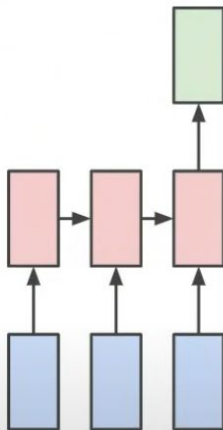
Sequential Data

One-to-one



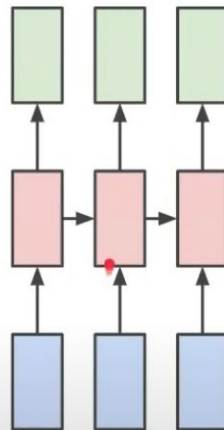
Vanilla Neural-net
Image classification

Many-to-one



Action recognition
(Frames $\rightarrow$ class)

Many-to-many



Frame classification
(Frames $\rightarrow$ classes)

One-to-many

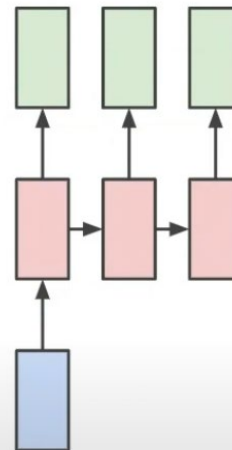
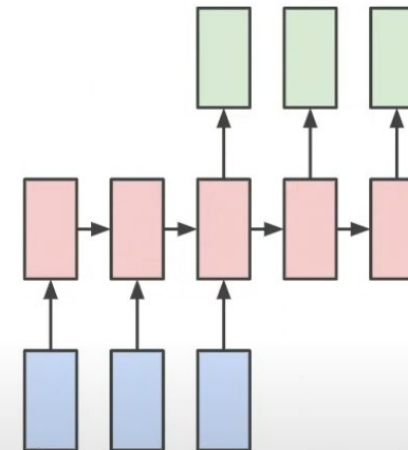


Image captioning
(Image $\rightarrow$ words)

Many-to-many

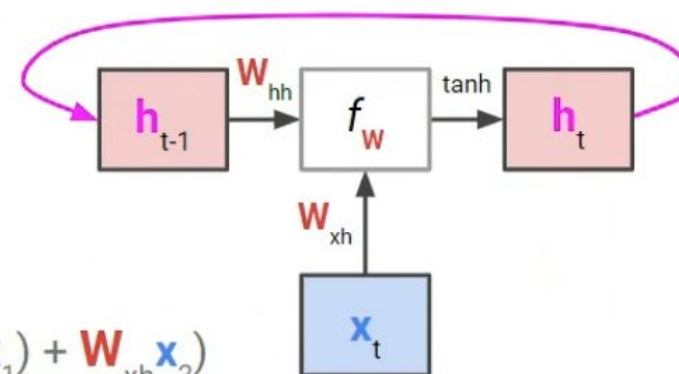


Video captioning
(Video $\rightarrow$ words)

RNN (Recurrent Neural Network)

Then, what about RNNs?

$$\mathbf{h}_t = f_{\mathbf{w}}(\mathbf{h}_{t-1}, \mathbf{x}_t) = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t)$$



$$\mathbf{h}_1 = \tanh(\mathbf{W}_{hh} \mathbf{h}_0 + \mathbf{W}_{xh} \mathbf{x}_1)$$

$$\mathbf{h}_2 = \tanh(\mathbf{W}_{hh} \mathbf{h}_1 + \mathbf{W}_{xh} \mathbf{x}_2) = \tanh(\mathbf{W}_{hh} \tanh(\mathbf{W}_{hh} \mathbf{h}_0 + \mathbf{W}_{xh} \mathbf{x}_1) + \mathbf{W}_{xh} \mathbf{x}_2)$$

$$\mathbf{h}_3 = \tanh(\mathbf{W}_{hh} \mathbf{h}_2 + \mathbf{W}_{xh} \mathbf{x}_3)$$

$$= \tanh(\mathbf{W}_{hh} \tanh(\mathbf{W}_{hh} \tanh(\mathbf{W}_{hh} \mathbf{h}_0 + \mathbf{W}_{xh} \mathbf{x}_1) + \mathbf{W}_{xh} \mathbf{x}_2) + \mathbf{W}_{xh} \mathbf{x}_3)$$

Again, the output $\hat{\mathbf{y}}$ is a weighted sum (+fixed unary operations) of input $\mathbf{x}$!

What is Transformer ?

A **Transformer** is a neural network architecture introduced in the 2017 paper “*Attention Is All You Need*” by Vaswani et al. It was designed for sequence-to-sequence tasks such as machine translation, and it has since become the foundation of most modern language models (e.g. BERT, GPT) and many vision models.

The key idea is to replace recurrent or convolutional operations with a mechanism called **self-attention**.

Transformer: Main Idea

- Basic assumption: the input x can be split into **multiple elements** that are **organically related** to each other.
 - People in a society
 - Words in a sentence
 - Frames in a video
 - ...
- **Self-attention**: Each element learns to refine its own representation by attending its **context** (other elements in the input).
 - More specifically, as a **weighted sum** of **other elements** (not arbitrary weights) in the sequence.

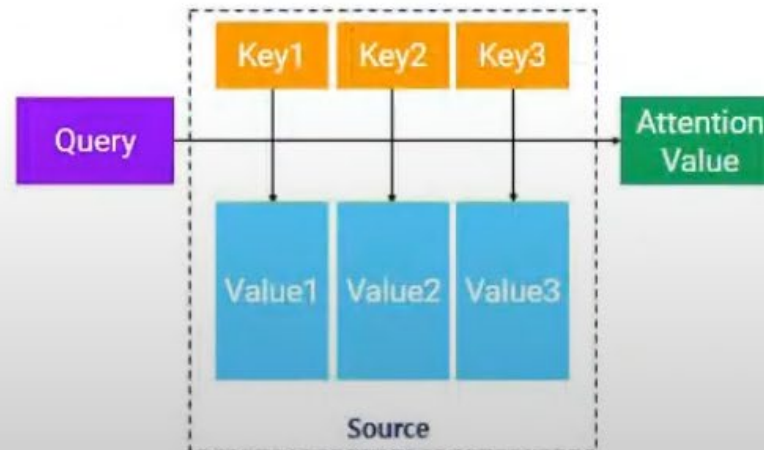


Attention (1)

- **Attention function:** $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Attention value}$

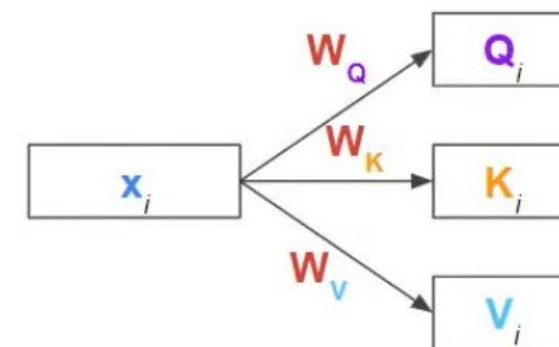
For a **query** (context) and **key-value** pairs (references), **attention value** is the weighted average of values, where each weight is proportional to the relevance between the query and the corresponding key.

- **Q** and **K** must be comparable (usually in the same dimensionality).
- **V** and **Attention value** are in the same dimensionality, obviously.
- In many applications, all four of these are in same dimensionality.



Attention (2)

- So, what should be the **Query**, **Key**, and **Value**?
- With Transformer, we **make** them!
 - From the input tokens $\{x_1, x_2, \dots, x_N\}$,
 - Each token x_i is mapped to its own **Query** Q_i , **Key** K_i , **Value** V_i vectors by a **linear transformation**.
 - The linear weights (W_Q, W_K, W_V) are the **learned parameters**, **shared by all inputs**.
 - W_Q (W_K, W_V) learns how to represent a vector to serve as a **Query** (**Key**, **Value**) in general.
- We need another learnable parameter, W_O , which maps the **attention value** back to the original space.



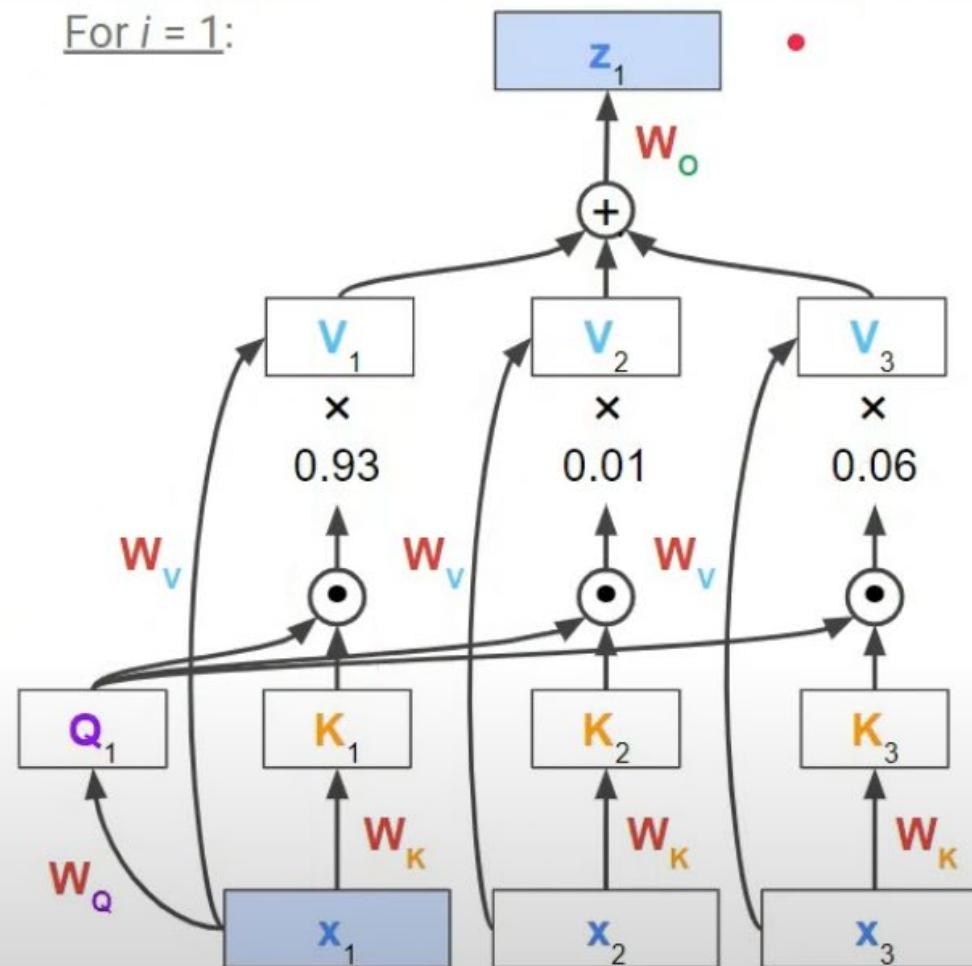
<https://arxiv.org/pdf/1706.03762.pdf>

Attention (3)

- Then, how do we perform attention?
 - Each token $\mathbf{x}_i$ becomes the **Query** when we learn about i .
 - *You are the main character in your life!*
 - References are all tokens $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ in the input sequence, including $\mathbf{x}_i$ itself.
 - *Your friends are a mirror that reflects you.*
- From this, we perform the attention:
 - Each element $\mathbf{x}_i$ is represented as a weighted (similarity computed using **Key**) sum of other elements (using **Value**) in $\mathbf{x}$.
 - $\mathbf{z}_i = \mathbf{w}_1 \mathbf{V}_1 + \dots + \mathbf{w}_N \mathbf{V}_N$, where $\mathbf{w}_j = \cos(\mathbf{Q}_i, \mathbf{K}_j)$
 - $\mathbf{W}_o$ maps from the **Value** space back to the original embedding space.

Attention (4)

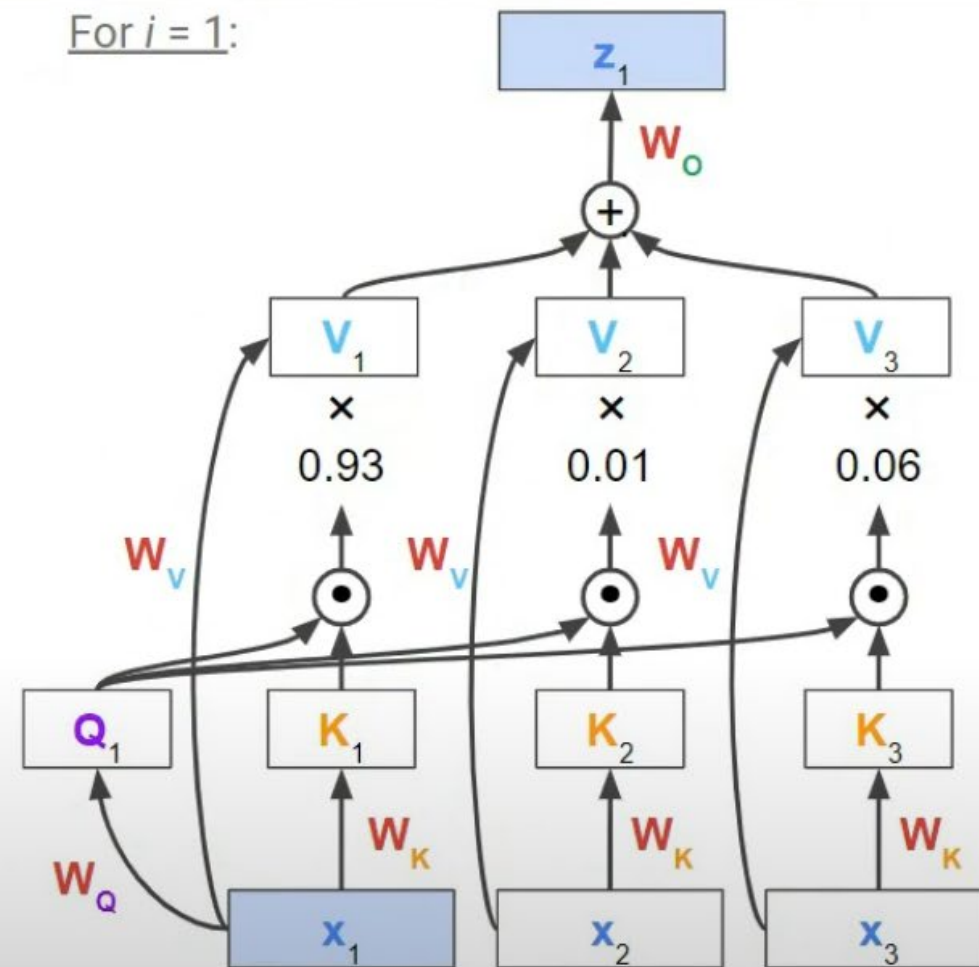
- Then, how do we perform attention?
 - Each token $\mathbf{x}_i$ becomes the **Query** when we learn about i .
 - *You are the main character in your life!*
 - References are all tokens $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ in the input sequence, including $\mathbf{x}_i$ itself.
 - *Your friends are a mirror that reflects you.*
- From this, we perform the attention:
 - Each element $\mathbf{x}_i$ is represented as a weighted (similarity computed using **Key**) sum of other elements (using **Value**) in $\mathbf{x}$.
 - $\mathbf{z}_i = \mathbf{w}_1 \mathbf{V}_1 + \dots + \mathbf{w}_N \mathbf{V}_N$, where $\mathbf{w}_j = \cos(\mathbf{Q}_i, \mathbf{K}_j)$
 - $\mathbf{W}_o$ maps from the **Value** space back to the original embedding space.



The same procedure is performed for all $i = 1, \dots, N$.

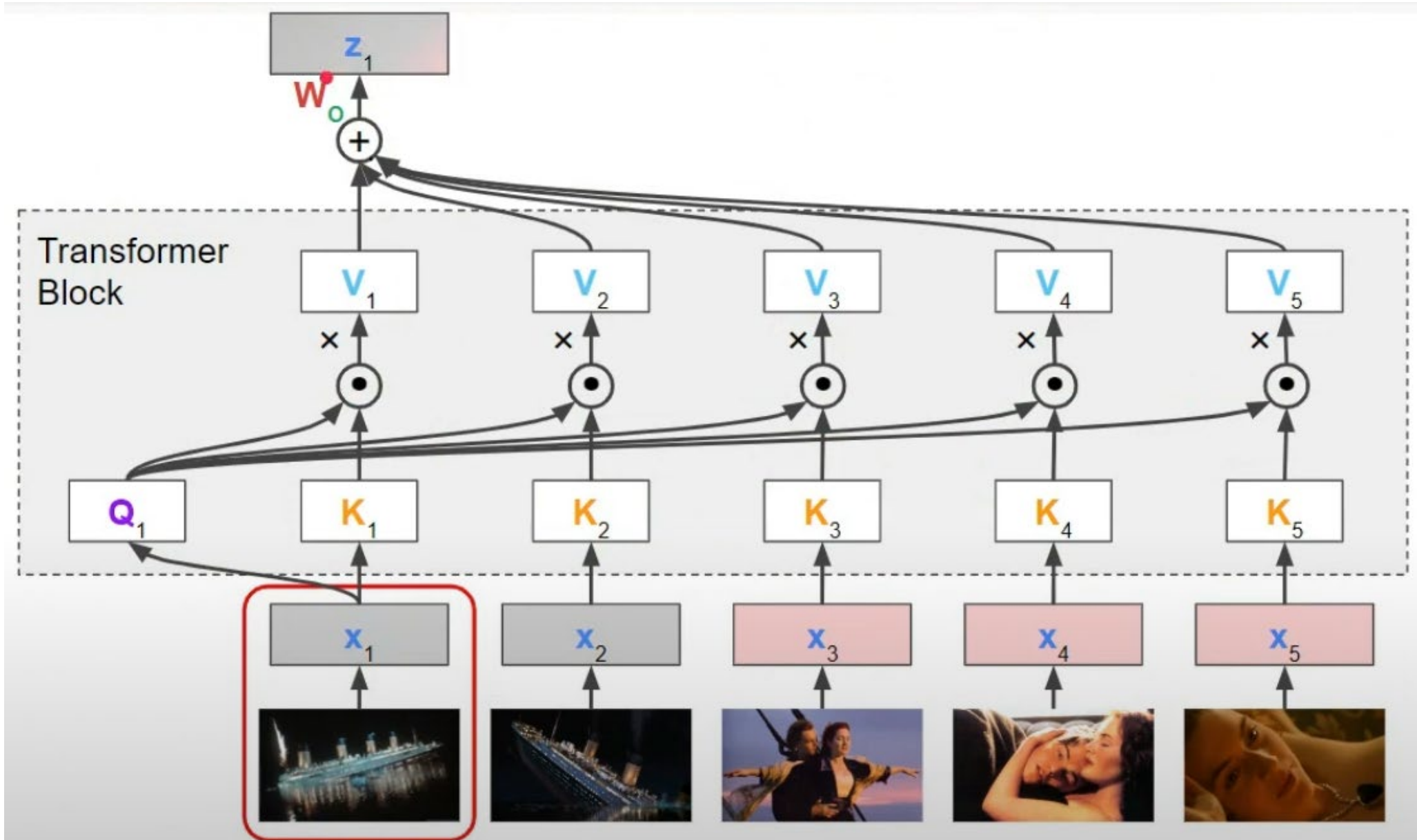
Attention (5)

- This resulting embedding $\mathbf{z}_1$ tends to be similar to its original one ($\mathbf{x}_1$), because $\cos(\mathbf{Q}_1, \mathbf{K}_1)$ is likely to be much higher than other $\cos(\mathbf{Q}_1, \mathbf{K}_i)$.
- The resulting $\mathbf{z}_1$ is still not exactly the same as the original one, slightly affected by its context (here, $\mathbf{x}_2, \mathbf{x}_3$).
- Usually, this step is repeated multiple times to further **contextualize**.

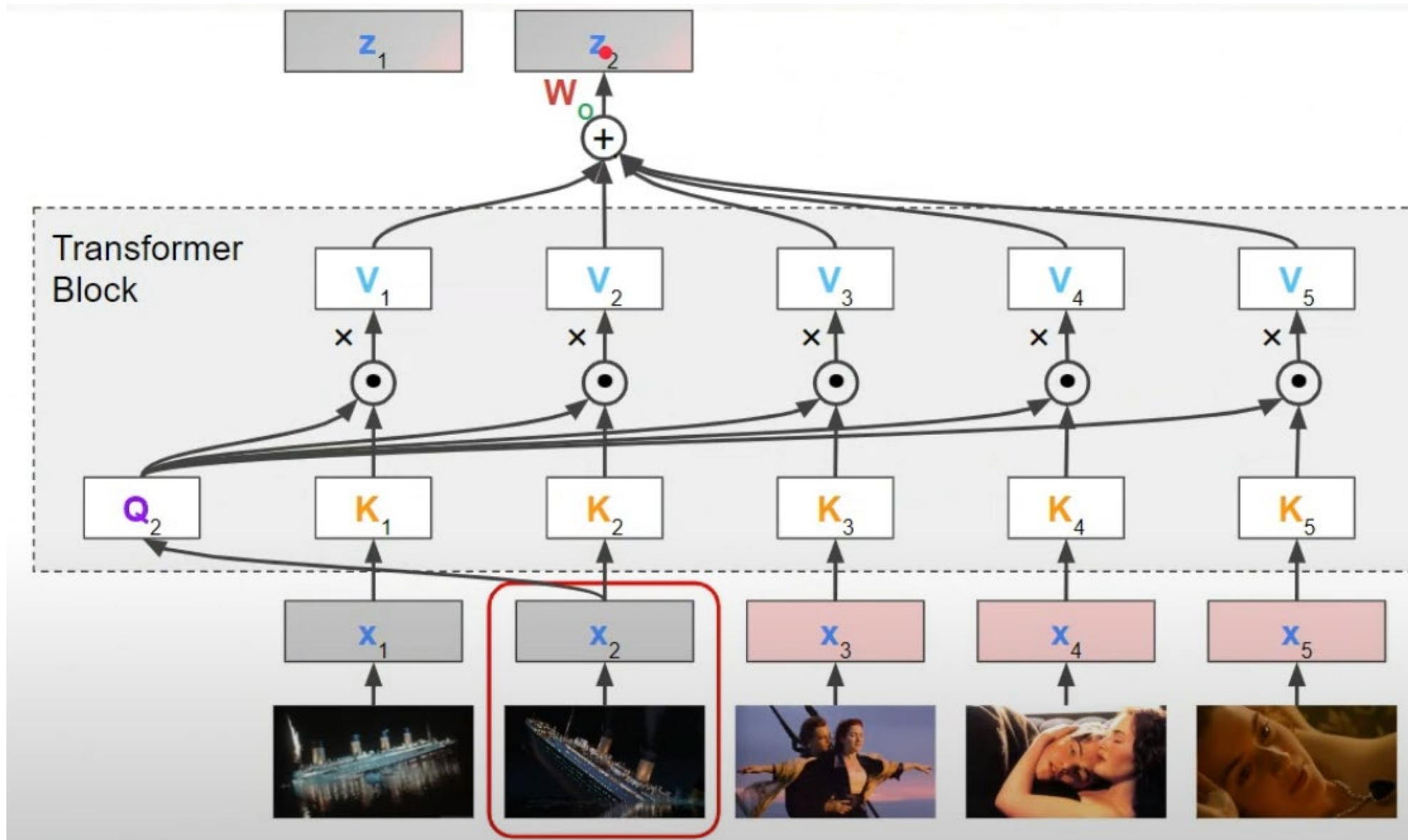


The same procedure is performed for all $i = 1, \dots, N$.

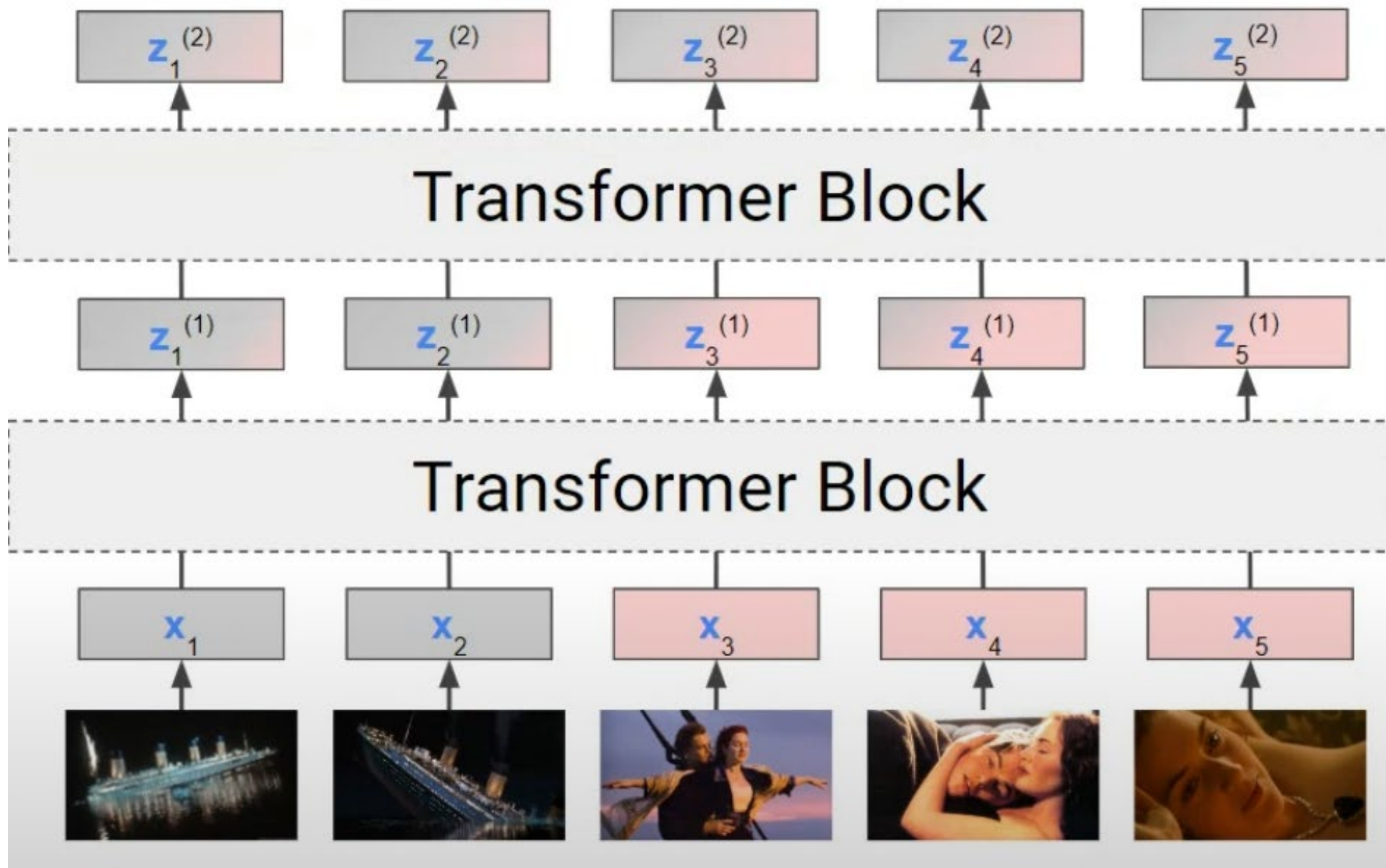
Attention (6)



Attention (7)



Attention (8)



What is Transformer ?

2. Self-Attention and Scaled Dot-Product Attention

2.1. Input Embeddings

Assume we have an input sequence of T tokens. Each token is mapped to a vector of dimension d_{model} . We can stack these into a matrix:

$$X \in \mathbb{R}^{T \times d_{\text{model}}}.$$

From X , the Transformer computes three matrices:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V,$$

where

$$W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d_k}.$$

Here,

- $Q = \text{queries}$
- $K = \text{keys}$
- $V = \text{values}$

What is Transformer ?

2.2. Scaled Dot-Product Attention

The **scaled dot-product attention** computes a weighted sum of the value vectors. For a set of queries Q , keys K , and values V , attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V.$$

Explanation:

- $QK^{\top}$ computes all pairwise dot products between queries and keys.
- The division by $\sqrt{d_k}$ is the *scaling*; it keeps the magnitude of the dot products from growing too large when d_k is large.
- softmax turns each row into a probability distribution over all positions.
- Multiplying by V computes a weighted sum of value vectors for each query.

This allows each token to attend to (i.e. focus on) other tokens in the sequence.

What is Transformer ?

3. Multi-Head Attention

Instead of using a single attention mechanism, Transformers use **multiple heads**.

Each head has its own projection matrices:

$$Q_h = XW_h^Q, \quad K_h = XW_h^K, \quad V_h = XW_h^V,$$

and produces:

$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h).$$

Then all heads are concatenated:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W^O,$$

where W^O is another learned projection matrix.

Intuition:

- Each head can focus on different types of relationships (e.g. syntax, long-range dependencies).
- Concatenating them allows the model to combine these different perspectives.

Thank you!

